FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Fritz Mardaniel L. Sablan
Section:	AN42
Professor:	Ms. Crisola G. Tan
Date:	02/10/2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition ----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition ----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input ----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}
```

```
# -------- Run Model ----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation ----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)

- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks)⇩

```python
import agentpy as ap
import random
import matplotlib.pyplot as plt

class RandomWalker(ap.Agent):

    def setup(self):
        self.path = []

    def step(self):

        directions = [
            (1, 0),
            (-1, 0),
            (0, 1),
            (0, -1)
        ]

        x, y = self.position

        possible_positions = []

        for dx, dy in directions:

            new_x = x + dx
            new_y = y + dy

            if (
                0 <= new_x < self.model.p.grid_size[0]
                and
```

```
            0 <= new_y < self.model.p.grid_size[1]
        ):
            possible_positions.append((new_x, new_y))

    occupied_positions = {
        agent.position
        for agent in self.model.agents
        if agent is not self
    }

    free_positions = [
        position
        for position in possible_positions
        if position not in occupied_positions
    ]

    if free_positions:
        self.position = random.choice(free_positions)
    else:
        self.position = (x, y)

    self.path.append(self.position)

class RandomWalkModel(ap.Model):

  def setup(self):

    self.agents = ap.AgentList(
        self,
        self.p.agents,
        RandomWalker
    )

    for agent in self.agents:

        agent.position = (
            random.randint(
                0,
                self.p.grid_size[0] - 1
            ),
            random.randint(
                0,
                self.p.grid_size[1] - 1
            )
```

```
            )

            agent.path = [agent.position]

    def step(self):

        for agent in self.agents:
            agent.step()

    def end(self):

        print("\nFinal Positions:")

        position_counts = {}

        for agent_number, agent in enumerate(
            self.agents,
            start=1
        ):

            print(
                f"Agent {agent_number}: {agent.position}"
            )

            if agent.position not in position_counts:
                position_counts[agent.position] = 0

            position_counts[agent.position] += 1

        print("\nFinal Position Distribution:")

        for position, count in position_counts.items():

            print(
                f"Position {position}: "
                f"{count} agent(s)"
            )

def run_simulation(
    scenario_name,
    num_agents,
    grid_size,
    num_steps
):
```

```
print("\n" + "=" * 60)
print(scenario_name)
print("=" * 60)

print(f"Number of agents: {num_agents}")
print(f"Grid size: {grid_size} x {grid_size}")
print(f"Number of steps: {num_steps}")

parameters = {
    "agents": num_agents,
    "grid_size": (grid_size, grid_size),
    "steps": num_steps
}

model = RandomWalkModel(parameters)

model.setup()

for step in range(num_steps):
    model.step()

model.end()

plt.figure(figsize=(8, 8))

for agent_number, agent in enumerate(
    model.agents,
    start=1
):

    x_values = [
        position[0]
        for position in agent.path
    ]

    y_values = [
        position[1]
        for position in agent.path
    ]

    plt.plot(
        x_values,
        y_values,
```

```
            marker="o",
            markersize=3,
            label=f"Agent {agent_number}"
        )

        plt.scatter(
            agent.position[0],
            agent.position[1],
            s=100
        )

    plt.xlim(-0.5, grid_size - 0.5)
    plt.ylim(-0.5, grid_size - 0.5)

    plt.xticks(range(grid_size))
    plt.yticks(range(grid_size))

    plt.grid(True)

    plt.title(
        f"{scenario_name}\n"
        f"Random Walk Agent Paths"
    )

    plt.xlabel("X Position")
    plt.ylabel("Y Position")

    plt.legend()

    plt.show()

    return model

print("=" * 60)
print("RANDOM WALK SIMULATION USING AGENTPY")
print("LABORATORY TASKS 1–4")
print("=" * 60)

scenario1 = run_simulation(
    scenario_name="Scenario 1",
    num_agents=5,
    grid_size=10,
    num_steps=20
)
```

```
scenario2 = run_simulation(
    scenario_name="Scenario 2",
    num_agents=10,
    grid_size=10,
    num_steps=50
)

print("\n" + "=" * 60)
print("SCENARIO COMPARISON")
print("=" * 60)

print("""
Scenario 1:
- 5 agents
- 10 x 10 grid
- 20 steps

Scenario 2:
- 10 agents
- 10 x 10 grid
- 50 steps

Observation:
Scenario 2 has more agents and more movement steps.
Therefore, there are more agent paths and more movement
throughout the grid compared with Scenario 1.

The path graphs can be used to visually compare how the
agents move and distribute themselves across the grid.
""")
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?
5.

Answer the Questions ⇩

1. If the number of agents is very high compared to the grid size, the agents become crowded and may occupy the same areas or positions. This makes the grid appear more congested, and it becomes more likely for agents to cluster together.

2. Increasing the number of steps gives the agents more opportunities to move around the grid. As a result, they generally explore more areas and may become more spread out over time, although some agents may still cluster in certain areas because their movement is random.

3. No. The agents cannot move outside the grid because the program restricts their positions to the grid boundaries. The max() and min() functions ensure that the x and y coordinates remain within the valid grid range.

4. How can this simulation be used to model real-life phenomena (e.g., diffusion, crowd movement)?
This simulation can be used to model real-life phenomena involving random movement. For example, it can represent diffusion, where particles spread randomly through an environment, or crowd movement, where individuals move around within a limited space. It can also be adapted to study other behaviors such as swarm movement and the distribution of entities over time.